REFERENCE ARCHITECTURE

The Edge Monolith

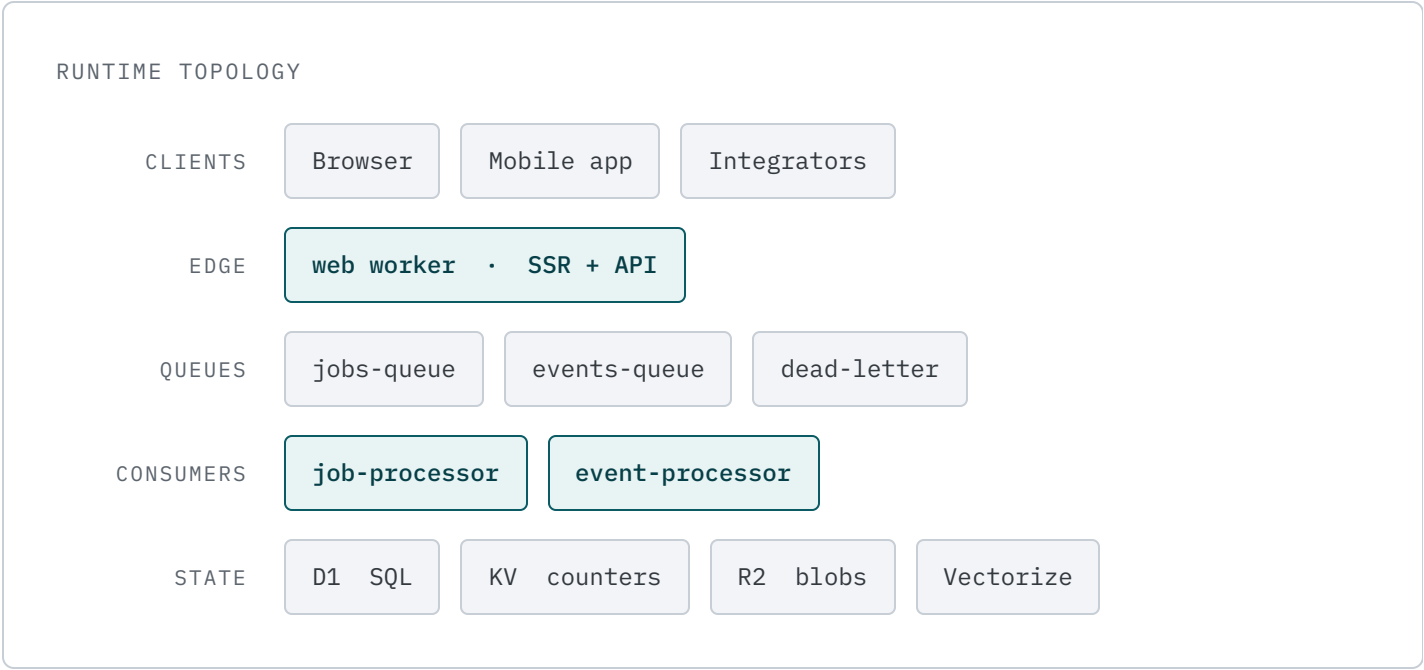
A full-stack pattern for Cloudflare Workers: one deployable that serves the website and the public API, a service layer that both entry paths share, and a small fleet of queue consumers behind it. Structured so the API can be split out later without touching a line of business logic.

Workers | D1 + Drizzle | Hono | Next.js / OpenNext | Queues

ORIENTATION

The shape of the thing

The premise: a consumer product with a web app, a public HTTP API that a mobile client and outside integrators consume, and a pile of asynchronous work that must not run in the request path. Conventional advice says three services. This pattern says **one Worker for anything user-facing, plus consumers for the async work** — and it gets away with it because the business logic lives in a layer neither entry point owns.



When this fits

- A product with one primary database and no need for independent scaling per domain.

- A team small enough that a service mesh is overhead, not leverage.
- Read-heavy traffic with modest, bounded write volume — the shape most consumer apps have.

When it does not

- Sustained high write throughput. SQLite at the edge has a single write primary; that ceiling is real.
- Domains needing genuinely separate deploy cadence or blast radius.
- Workloads needing long-running compute — Workers are request-scoped, with CPU-time limits.

REPOSITORY

Package layout

A `pnpm` workspace. The critical move is that **the API is a library, not a Worker** — it has no deploy target of its own. Every runtime that needs business logic imports it and supplies its own context.

```
# packages/
api/                library — services, controllers, schema, policies
web/                deployable — Next.js + mounted HTTP API
job-processor/      deployable — queue consumer
event-processor/    deployable — queue consumer
```

Give `packages/api` a granular `exports` map rather than a single barrel. The frontend imports validation schemas, DTO types, and constants directly from it; a fine-grained export map is what stops the bundler from pulling server-only code into a browser bundle.

```
// packages/api/package.json
"exports": {
  ".":                  "./src/index.ts",
  "./schemas/order":    "./src/schemas/order.ts",    // shared with forms
  "./services/dto/*":   "./src/services/dto/*.ts",   // wire types
  "./policies/actor":   "./src/policies/actor.ts"
}
```

> WHY IT PAYS OFF
>
> One Zod schema validates the browser form and the API request. One DTO type describes what the endpoint returns and what the component renders. Contract drift between frontend and backend stops being a category of bug.

CORE PATTERN

The service factory

This is the keystone. Every unit of business logic is a class with a **private constructor** and a **static** `make(ctx)` factory that declares exactly the dependencies it needs. Nothing injects a god-object context; nothing reaches for a global.

```
@Service("orderService")
export class OrderService {
  private constructor(
    private db: Database,
    private pricing: PricingService,
  ) {}

  // The ONLY construction path. Compose ctx from your deps' ctx types,
  // so removing a dependency narrows the type and the compiler finds
  // every now-unnecessary field for you.
  static make(
    ctx: Parameters<typeof PricingService.make>[0] & { var: { db: Database } },
  ) {
    return new OrderService(ctx.var.db, PricingService.make(ctx));
  }

  async place(actor: Actor, input: OrderInput) { /* … */ }
}
```

The four rules

RULE	REASON
Private constructor, explicit deps	Makes the dependency surface visible and testable. Tests call `make({ var: { db: fake } })` — no framework, no container, no fake request.
Services are stateless	Worker isolates are recycled unpredictably. Instance caches produce bugs that only reproduce under load. Use a `static` field if you truly need a process-wide cache.
Compose ctx types from deps	The context type is derived, never hand-maintained. Refactors stay honest.
Name the decorator explicitly	Minifiers mangle class names. A bare `@Service()` that registers under `target.name` resolves to a hash in production and the lookup silently returns undefined.

HARD RULE

`@Service("orderService")` — always pass the name string. This is the single most expensive mistake to debug in this pattern, because it works perfectly in development and fails only on the minified deploy.

Codegen closes the loop

A prebuild step scans for decorated classes and writes two artifacts: barrel files (so importing the barrel executes the decorators and populates the registry) and a typed registry via TypeScript module augmentation. The payoff is `make.orderService.place(…)` — fully typed, no runtime container.

```
# package.json
"prebuild": "node scripts/gen-barrels.mjs && node scripts/gen-factory-types.mjs",
"typecheck": "pnpm prebuild && tsc --noEmit"
```

Generated files are gitignored and every script that needs them runs `prebuild` first. A fresh clone will not typecheck until it has run — document that loudly in the README, because the error message (`Cannot find module './services'`) does not suggest the fix.

BACKEND

Layers, and what belongs in each

Six layers. The discipline is not that they exist — most codebases claim layers — but that each one has a rule about what it may *not* do.

REQUEST → RESPONSE

route	Parse and validate input, call one controller method, shape the response. **No business logic, no database access.**
middleware	Cross-cutting concerns: session resolution, rate limits, bot checks, origin checks, request-scoped DB client. **Never the sole place authorization happens.**
controller	Orchestrates a use case across services. **Enforces scopes and ownership itself**, taking an `Actor` — never a bare user id.
service	The business logic. Stateless, constructed through `make()`. **Owns all database access.**
policy	Pure authorization predicates — `(actor, resource) → allowed | denied`. No I/O, trivially unit-testable.
dto	Explicit wire shapes: column omission, computed fields, join flattening. **Rows are never serialized directly.**

The actor

Model the caller as a discriminated union, not a nullable user id. Anonymous is a first-class case, so "logged out" can never be confused with "logged in but no permissions".

```
export type Actor =
  | { kind: "anonymous" }
  | {
      kind: "user";
      user_id: string;
      scopes: Scope[];
      memberships: Membership[];   // org/team access
      restricted: boolean;         // suspended, banned, etc.
    };
```

The error envelope

Fix one response shape for every failure, everywhere. Codes are machine-readable, `SCREAMING_SNAKE_CASE`, and never localized — the client maps a code to a translated string it owns.

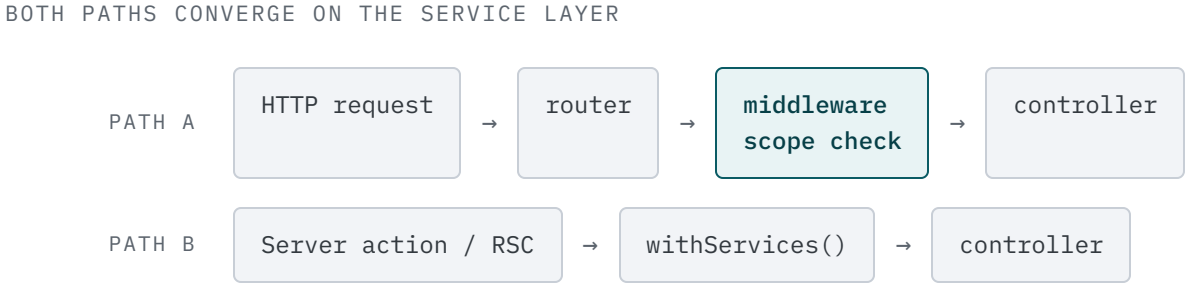
```
{
  "error": {
    "code": "VALIDATION_FAILED",
    "detail": {
      "issues": [{ "path": "email", "message": "INVALID_EMAIL" }]
    }
  }
}
```

Emit field issues through a single shared helper. The moment two routes serialize validation errors differently, the client-side parser that maps issues onto form fields silently stops working for one of them — with no error to trace.

CRITICAL

Two call paths, one enforcement point

This is where the pattern's main security hazard lives, and it deserves its own section. Business logic is reachable two ways:



Path B never touches the HTTP router — so **route middleware does not run for it**. Any check that lives only in middleware is absent on the in-process path. A layout that verifies "user has some admin scope" does not verify the specific verb the action performs.

> THE RULE THAT PREVENTS THE BUG
>
> **Enforce scopes inside the service or controller method, never only on the route.** The method takes the `Actor` and checks it before any mutation. Keep the route middleware too — defense in depth and a clean 403 envelope — but treat it as a second line, not the line.

```
private requireScope(actor: Actor, scope: Scope): string {
  if (actor.kind !== "user" || !actor.scopes.includes(scope)) {
    throw new AppError(ErrorCode.FORBIDDEN, 403);
  }
  return actor.user_id;  // hand back for audit logging
}
```

The tell for an unguarded method is its signature. A method typed `(id: string)` or `(userId, actorId: string)` that performs a write is trusting its caller. If a server action calls it, it is unguarded on the web path. Fix it at the method, not the call site.

The in-process bridge

Path B needs an adapter that mirrors what the HTTP middleware chain builds: bindings, a request-scoped database client, locale, and a resolved actor.

```
export async function withServices<T>(
  fn: (make: TypedFactory, ctx: Ctx) => Promise<T>,
): Promise<T> {
  const { env, ctx: executionCtx } = getCloudflareContext();
  const db = drizzle(env.DB);
  const actor = await resolveActor(env);   // mirrors optionalSession
  return fn(ServiceFactory.createTyped({ env, var: { db }, actor }), ctx);
}
```

TWO DETAILS WORTH STEALING

Race the session lookup against a timeout and fall back to anonymous. Public pages resolve the actor for analytics but do not need it to render; a hung session query should not hold the page hostage. Falling back to anonymous is fail-safe — scope checks then see no scopes and deny.

Fail closed on a missing environment flag. If `ENVIRONMENT` is absent, resolve it to production rather than letting downstream code read `undefined` and quietly take the development branch — the one that stubs payments and skips verification.

FRONTEND

The web layer

Next.js App Router compiled to a Worker via the OpenNext Cloudflare adapter. The HTTP API mounts inside it as a catch-all route, so one deploy serves both.

```
app/
  api/[[...route]]/route.ts   // the whole HTTP API mounts here
  [locale]/
    (marketing)/              // public, crawlable
    (app)/                    // authenticated product
    (auth)/                   // sign-in, verification
  (admin)/admin/              // internal console, scope-gated
```

Rendering rule

Any page reaching for platform bindings must opt out of static prerendering. The context accessor is synchronous and there is no request context at build time, so the build aborts while prerendering.

```
export const dynamic = "force-dynamic";   // first line of the file

import { getSession } from "@/lib/auth";
```

Make it a convention applied uniformly rather than a per-page judgment call. It is the kind of rule that is invisible until a build fails at deploy time.

Composition discipline

- **Extract on the third copy, not the second.** Two similar blocks are fine; three is the point at which the next contributor will cargo-cult a fourth.
- **Share schemas, don't restate them.** The form resolver consumes the same Zod schema the endpoint validates against.
- **Normalize on the server.** When a form's empty string means "clear this field", convert it to null in the API schema, not in each client. Web, mobile, and hand-written requests then behave identically.

Internationalization from day one

Retrofitting i18n is expensive; adopting it while the app is small is nearly free. Route with a `[locale]` segment, and use CSS logical properties throughout — `margin-inline-start` over `margin-left`, `text-align: start` over `left`. Enforce it with a lint rule that fails the build on directional utilities. If a right-to-left language ever ships, the layout already works.

DATA

Database and migrations

D1 is managed SQLite: one database per environment, reached through a binding rather than a connection string. Drizzle gives typed queries and generates migrations. Mirror your wire casing in the schema — if the API speaks `snake_case`, name the columns `snake_case` and no translation layer is needed.

NEVER RECREATE A TABLE

Migration generators emit a "copy into `__new_table`, drop, rename" dance for anything SQLite cannot express as a plain `ALTER` — adding a CHECK constraint, relaxing NOT NULL, changing a primary key. **Delete that and hand-write the alternative.**

It fails on D1 specifically, because each statement runs in its own implicit transaction: `PRAGMA foreign_keys=OFF` resets before the `DROP`, which then trips every foreign key pointing at the table. It is also dangerous in general — it copies every row under a hold and leaves a half-migrated database if any statement fails.

CHANGE	DO THIS INSTEAD
Add / drop a column	`ALTER TABLE t ADD COLUMN c text;`
Relax NOT NULL	Stage values in a side column, alter, restore.
Add CHECK to an existing table	Don't. Enforce in the request schema and narrow at the DTO boundary on read.
Add CHECK to a new table	Fine — the recreate only fires when the table already exists.

Naming

Require a description on every generated migration. Default generators produce names like `0007_curious_silver_falcon.sql`, which is useless in a log, useless when triaging a failure, and useless in review.

```
pnpm db:generate --name add_order_status_index   ✓
pnpm db:generate                                 ✗ reject in review
```

Enforce it with a prebuild check so the rule is mechanical rather than social.

ASYNCHRONOUS WORK

Queues and the consumers

Get the mental model right: **database writes are synchronous, in the request path.** A user places an order, the row commits, the response returns. Queues carry the *side effects* — the work that must not block the response and needs retry semantics.

IN-REQUEST	QUEUED
Domain writes the user must see reflected immediately	Notifications — email, SMS, push
Reads that render the response	Third-party sync (CRM, billing, search index)
Authorization and validation	Analytics events
	Batch or AI jobs measured in seconds

Consumer configuration

```
# wrangler.toml — the consumer Worker
[[queues.consumers]]
queue             = "jobs-{env}"
dead_letter_queue = "dlq-{env}"
max_batch_size    = 5
max_retries       = 10

# Consume the DLQ too — with no retries. Poisoned messages get
# logged and acked once instead of cycling forever.
[[queues.consumers]]
queue          = "dlq-{env}"
max_retries    = 0
```

Handlers dispatch on an event `type` field over a discriminated union, so adding an event type is a compile error until every consumer handles it.

> **THE SILENT-DROP TRAP**
>
> Enqueueing inside `waitUntil()` is fire-and-forget: if the binding is missing, it no-ops with no error. For events that genuinely matter, do a **direct, awaited send** and require every call site to await it. A non-awaited send can be lost when the isolate is evicted, and it leaves no trace.
>
> One deliberate exception: paths that must return in constant time — verification-code senders, where response timing would leak whether an account exists. There, wrap the send in `waitUntil()` so the runtime still completes it after the response is sent.

IDENTITY

Authentication

Use a session library as the single source of truth rather than minting your own tokens: cookies for browsers, `Authorization: Bearer` for mobile and integrators, both resolving to the same session record. Store sessions in the same database as everything else.

Verify at the app layer, not the auth layer

Sign-up mints a session immediately; email verification is enforced by your own gate. This sounds like a small distinction and is not. It is what makes the multi-device flow work: sign up on a laptop, tap the verification link on a phone, and the laptop session unblocks on its next request — no second sign-in.

```
type SessionState =
  | { state: "unauthenticated" }
  | { state: "unverified"; user: User }
  | { state: "verified";   user: User };
```

Trust a cached `verified` (the flag is monotonic — it never goes back), and re-check the database when the cached value is `false`. That keeps the common path free of a query while making the transition immediate.

Scopes

Store scopes on the session and expose them on the actor. Keep them coarse and verb-shaped — `admin.users.write`, `admin.audit.read` — and remember the rule from earlier: the method enforces them, not just the route.

INSTRUMENTATION

Analytics that stay trustworthy

Emit events server-side into a queue, enrich them in one place, and let a consumer forward them to whichever product-analytics backend you use. Client-side SDKs are blocked, batched away, and inconsistent across browsers; server-emitted events are not.

01 **Define events as a typed union.**

One module per domain area, each event a named shape with an explicit payload. Adding a property is a type change, so no consumer silently misses it.

02 Enrich centrally, once.

A single function attaches device, geo, attribution, consent, and bot signals — parsed from the request, coarsened deliberately. Backend-emitted events pass nulls rather than fabricating a device.

03 Gate on consent at the source.

Read the consent cookie during enrichment and drop or degrade the event there, so no downstream consumer has to remember the rule.

04 Enqueue, never block.

The request path pays a queue send, not a network round trip to a vendor.

05 **Forward in a consumer.**

Retries,
batching,
and
vendor
outages
are
the
consumer's
problem.
Swapping
vendors
touches
one
handler.

Coarsen deliberately. Major browser and OS version, no patch numbers, no device model. It is enough for product decisions and materially reduces how identifying the event stream is.

Errors and observability

Ship an error tracker with a **scrubbing module you own** — a deny-list applied before send, covering tokens, emails, phone numbers, and any field your domain treats as sensitive. Default SDK scrubbing is not enough. Wrap the database client so query spans appear on traces without instrumenting call sites.

DEVELOPER EXPERIENCE

The local loop

Target: clone, install, two commands, running app — with no cloud account required for ordinary work. Everything except vector search has a local emulator.

```
pnpm install
pnpm db:migrate:local          # creates the local SQLite file
pnpm dev                       # web + both consumers, in parallel
```

Run the consumers with `--persist-to` pointed at the web package's state directory. All three processes then share one local database, one KV store, and one set of queues — so a queued event produced by the web app is actually consumed locally. Without this they each get a private, empty world and the integration never runs until staging.

Two env files, and the difference matters

FILE	HOLDS	READ BY
`.dev.vars`	Server secrets — signing keys, API tokens	The Worker runtime, as `env.NAME`
`.env.local`	Public values inlined into the browser bundle	The bundler, at build time

Commit an annotated `.example` for each, gitignore the real ones, and say in the comments what each value does, whether it is optional, and what happens when it is blank. This file is the onboarding document people actually read.

> **DEGRADE, DON'T FAIL**
>
> Make every third-party integration no-op or stub when its credential is absent: payments return a fake checkout URL, email logs the message and its links to the console, CRM sync returns early. A new contributor should reach a working signup flow with zero external accounts. Reserve real credentials for the specific feature someone is actively working on.

> **REMOTE-ONLY BINDINGS**
>
> Vector indexes have no local emulator; the binding proxies to a real dev-only index and therefore requires an authenticated CLI session. Give that a documented fallback — a config flag that disables the remote binding and degrades to keyword-only search — so a developer who has not logged in can still run everything else. Put the fallback in a comment beside the binding, not in a wiki.

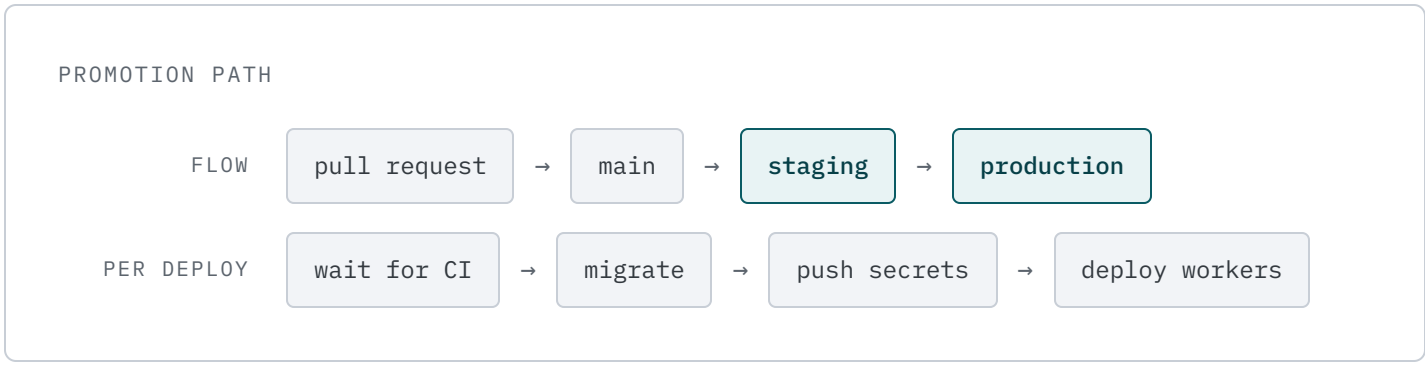
Editing environment values

Server secrets are re-read when the dev server restarts. Public build-time values are inlined into the bundle, so changing one requires a restart *and* a rebuild — the running process keeps serving the old value with no warning. Worth a line in the README; it costs people an afternoon otherwise.

DELIVERY

Deployment

Branch-driven promotion. A commit is verified once and that exact SHA moves forward — no rebuilds, no drift between what was tested and what ships.



Gate on CI, but only run it once

The deploy's first job polls for the test and typecheck checks *on the same SHA* and refuses to proceed without them. Pair that with a duplicate-action skip so a tree already verified on the pull request is not rebuilt when the same tree lands on `main` and then `staging`. Accept both `success` and `skipped` as a pass. One pull request, one real CI run, and the deploy still hard-gates on green.

Declare secrets, don't script them

Keep one manifest describing which secrets each Worker receives. The deploy reads it and bulk-uploads; adding a secret is a one-line change with no workflow edit.

```yaml
# .github/worker-secrets.yml
workers:
  web:
    dir: packages/web
    secrets:
      SESSION_SECRET: SESSION_SECRET      # worker name ← GH secret name
      SENTRY_DSN:     SENTRY_DSN_WEB      # per-app alias
    optional:
      - ANALYTICS_TOKEN                   # missing warns, doesn't fail
```

Four homes for configuration, and choosing wrong is a slow leak:

KIND	HOME
Secrets — tokens, keys	Manifest `secrets:`, from environment-scoped store
Non-secret, same everywhere	Manifest `vars:` as a literal
Non-secret, differs per environment	Manifest `secrets:` — the per-env store is the point, not the sensitivity
Bindings and infra wiring	`wrangler.toml` — also feeds local dev

KIND	HOME
Build-time public values	Manifest `build_env:`, resolved before the bundler runs

Make anything not explicitly marked optional **required**, so a missing value fails the deploy loudly instead of letting an environment drift into a half-configured state that surfaces days later.

Migrations run before the deploy

A dedicated job applies migrations, then the Workers deploy in parallel. Because the rule above forbids table recreation, migrations are additive and a new schema is compatible with the currently-running code for the seconds between the two steps.

FIELD NOTES

Pitfalls, ranked by cost

These are the failures that cost the most to diagnose — mostly because each one works perfectly in development.

SYMPTOM	CAUSE	FIX
Privilege escalation via a server action	Scope checked in route middleware only; the in-process path skips it	Enforce in the method, taking the `Actor`
Works locally, undefined service in production	Bare `@Service()`; minifier mangled the class name	Always pass the explicit name string
Events vanish under load	Non-awaited queue send lost on isolate eviction	Await every send; `waitUntil` only for constant-time paths
Migration fails on deploy, fine locally	Generated table-recreate; per-statement transactions defeat the FK pragma	Hand-write `ALTER`; never ship `__new_table`
Build aborts prerendering a page	Platform context accessed at build time	`export const dynamic = "force-dynamic"`
Field-level form errors silently stop showing	A route serialized validation issues in a different shape	One shared serializer; make the wire shape unmissable
Fresh clone will not typecheck	Generated barrels and registry types absent	Run prebuild from every script that needs it

SYMPTOM	CAUSE	FIX
Env change appears to do nothing	Public value was inlined into the bundle at build time	Restart and rebuild; document the distinction
Repository clone is slow and growing	Binary assets committed; history keeps blobs after deletion	Serve from object storage behind a CDN; purge from history, not just `rm`

Two conventions worth adopting on day one

- **Binary assets never enter the repository.** Images, fonts, video, audio go to object storage behind a CDN and are referenced through one helper that prefixes the base URL. Git keeps deleted blobs forever, so this is far cheaper to prevent than to fix.
- **Money is never a float.** Pick an arbitrary-precision decimal library, wrap it once with your own division and comparison helpers, and import only the wrapper — so rounding policy has exactly one home.

A pattern, not a prescription. The parts worth copying wholesale are the service factory, the actor-based authorization rule, and the secrets manifest — each one prevents a class of bug rather than merely organizing code. The rest should bend to your domain.